

Licenciatura en Ingeniería en Tecnologías de la Información e Innovación Digital

Desarrollo de Software Multiplataforma

Estructura de Datos

UNIDAD II

Estructuras de datos básicas

Pilas y Colas

Hernández Torrez Alondra Vianney -1224100684

Grupo: GTID 141

Docente:

Gabriel Barrón Rodríguez



Dolores Hidalgo. C.I.N. Gto, Martes 4 de Noviembre de 2025.

Ejercicios Elegidos

Pilas RÚBRICA

Criterio	Excelente (100%)
Implementación del código	Código completo, sin errores y funcional
Documentación y comentarios	Documentación clara y completa
Funcionamiento del programa a través de video	Cumple con todos los requisitos
Probar tú Código con Debug	Excelente formato y organización

2. Pila de nombres

Objetivo: Permitir al usuario ingresar nombres y mostrarlos en orden inverso.

Pseudocódigo:

Algoritmo:

1. Leer nombres hasta escribir 'FIN'
2. Apilar cada nombre
3. Desapilar e imprimir en orden inverso

Implementación en Java con tú código:

```
import java.util.Scanner;
import java.util.Stack;
public class Ejercicio2 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Stack<String> pila = new Stack<>();
        String nombre;
        while (true) {
            System.out.print("Ingrese un nombre (FIN para salir): ");
            nombre = sc.nextLine();
            if (nombre.equalsIgnoreCase("FIN")) break;
            pila.push(nombre);
        }
    }
}
```

```
System.out.println("Nombres en orden inverso:");
while (!pila.isEmpty()) {
    System.out.println(pila.pop());
}
}
```

Ejemplo de salida: Entrada: Ana, Luis, Pedro, FIN → Salida: Pedro, Luis, Ana

Pseudocódigo

Inicio

 Crear una pila vacía para guardar nombres

 Pedir al usuario que escriba un nombre

 Mientras el nombre no sea "FIN"

 Guardar el nombre en la pila

 Pedir otro nombre

 Fin Mientras

 Mostrar el mensaje "Nombres en orden inverso:"

 Mientras la pila tenga nombres

 Sacar el nombre de arriba de la pila

 Mostrarlo en pantalla

 Fin Mientras

Fin

Algoritmo paso a paso

1. Se crea una pila vacía que almacenará los nombres.
2. Se solicita al usuario que escriba un nombre.
3. Mientras el nombre no sea "FIN", se guarda (apila) en la pila.
4. Cuando el usuario escriba "FIN", el programa termina la entrada de datos.
5. Luego, el programa empieza a desapilar los nombres uno por uno y los muestra en pantalla.
6. El resultado final es la lista de nombres en orden inverso al que se ingresaron.

4. Invertir una palabra

Objetivo: Invertir una palabra usando una pila de caracteres.

Pseudocódigo:

Algoritmo:

1. Leer palabra
2. Apilar cada letra
3. Desapilar e imprimir

Implementación en Java:

```
import java.util.Stack;
import java.util.Scanner;
public class Ejercicio4 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Ingrese una palabra: ");
        String palabra = sc.nextLine();
        Stack<Character> pila = new Stack<>();
        for (char c : palabra.toCharArray()) pila.push(c);
        System.out.print("Invertida: ");
        while (!pila.isEmpty()) System.out.print(pila.pop());
    }
}
```

Ejemplo de salida: Entrada: UTNG → Salida: GNTU

Pseudocódigo

Inicio

- Mostrar mensaje: "Ingrese una palabra"
- Leer la palabra escrita por el usuario
- Crear una pila vacía para guardar los caracteres

Para cada letra en la palabra

- Apilar la letra en la pila

Fin Para

Mostrar el mensaje "Palabra invertida: "

Mientras la pila no esté vacía

- Sacar (desapilar) la letra de la pila

- Mostrar la letra sin salto de línea

Fin Mientras

Fin

Algoritmo paso a paso

1. Se solicita al usuario que escriba una palabra.
2. Se crea una pila de tipo carácter (char).

3. Cada letra de la palabra se apila una por una.
4. Luego, se desapilan las letras y se imprimen sin salto de línea.
5. Esto genera la palabra invertida, porque la pila devuelve los elementos en orden LIFO (último en entrar, primero en salir).

6. Decimal a binario

Objetivo: Convertir número decimal a binario usando una pila.

Pseudocódigo:

Algoritmo:

1. Dividir número por 2 y apilar restos

2. Mostrar elementos desde la pila

Implementación en Java:

```
import java.util.Stack;
import java.util.Scanner;
public class Ejercicio6 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Ingrese número: ");
        int num = sc.nextInt();
        Stack<Integer> pila = new Stack<>();
        while (num > 0) {
            pila.push(num % 2);
            num /= 2;
        }
        System.out.print("Binario: ");
        while (!pila.isEmpty()) System.out.print(pila.pop());
    }
}
```

Ejemplo de salida: Entrada: 13 → Salida: 1101

Pseudocódigo

Inicio

Mostrar mensaje: "Ingrese un número decimal"

Leer el número

Crear una pila vacía para guardar los restos

Mientras el número sea mayor que 0

 Calcular el residuo de dividir entre 2

 Apilar el residuo

 Dividir el número entre 2 (solo la parte entera)

Fin Mientras

Mostrar mensaje: "Número en binario:"

Mientras la pila no esté vacía

Desapilar el número de arriba

Mostrarlo sin salto de línea

Fin Mientras

Fin

Algoritmo paso a paso

- ★ Se pide al usuario que ingrese un número decimal.
- ★ Se crea una pila de tipo entero para almacenar los restos de cada división entre 2.
- ★ Mientras el número sea mayor a 0:
 - Se obtiene el residuo de dividir el número entre 2 (0 o 1).
 - Ese residuo se apila.
 - El número se divide entre 2 (entero).
- ★ Al terminar, los restos guardados en la pila representan el número binario, pero en orden inverso.
- ★ Se desapilan los restos uno por uno para mostrarlos en el orden correcto (de arriba hacia abajo).

3. Verificar si una pila está vacía

Objetivo: Comprobar si la pila tiene elementos.

Pseudocódigo:

Algoritmo:

1. Crear pila vacía

2. Usar método isEmpty()

Implementación en Java:

```
import java.util.Stack;
```

```
public class Ejercicio3 {
```

```
    public static void main(String[] args) {
```

```
        Stack<Integer> pila = new Stack<>();
```

```
System.out.println("¿Está vacía la pila? " + pila.isEmpty());
pila.push(1);
System.out.println("¿Está vacía la pila? " + pila.isEmpty());
}
}
```

Ejemplo de salida: Salida esperada: true, false

Pseudocódigo

Inicio

- Crear una pila vacía
- Comprobar si la pila está vacía usando isEmpty()
- Mostrar el resultado
- Agregar (apilar) un número en la pila
- Volver a comprobar si la pila está vacía
- Mostrar el resultado nuevamente

Fin

Algoritmo paso a paso

1. Se crea una pila vacía de tipo entero.
2. Se usa el método isEmpty() para verificar si la pila tiene elementos.
3. Se muestra el resultado (debe ser "true", porque está vacía).
4. Se apila un número dentro de la pila.
5. Se vuelve a usar isEmpty() para comprobar su estado.
6. Ahora debe mostrar "false", porque ya contiene un elemento.



Ejercicio 1: Comparación de colas

Descripción:

Escribir un método que reciba como argumentos dos colas del mismo tipo y devuelva true si ambas colas son idénticas (mismo tamaño y mismos elementos en el mismo orden), o false en caso contrario.

Objetivos:

- Aplicar estructuras de datos tipo cola.
- Implementar recorrido y comparación de elementos.
- Preservar el estado original de las colas (opcional con colas auxiliares).

Pseudocódigo

Inicio

Crear dos colas vacías
Pedir al usuario cuántos elementos tendrá cada cola
Leer los elementos de la primera cola
Leer los elementos de la segunda cola
Mostrar las dos colas

Si las colas no tienen el mismo tamaño

Mostrar que no son iguales

Si no

Comparar elemento por elemento quitando de ambas colas

Guardar en colas auxiliares para no perder los datos

Si todos los elementos son iguales → devolver true

Si alguno es distinto → devolver false

Al final restaurar las colas originales con los auxiliares

Fin Si

Fin

Algoritmo

1. Se crean dos colas vacías (LinkedList se usa para representar colas en Java).
2. El usuario elige cuántos elementos tendrán las colas.
3. Se piden los valores de cada cola desde teclado.
4. Se muestran ambas colas.
5. Si los tamaños son diferentes, ya sabemos que no son iguales.
6. Si tienen el mismo tamaño, se comparan elemento a elemento.

7. Se guardan los valores en colas auxiliares para restaurarlas después (así no se destruyen).
8. Al final, se regresa si las colas son iguales o no.